Claude Code as a Research Group

A practical tour

Cameron F. Abrams

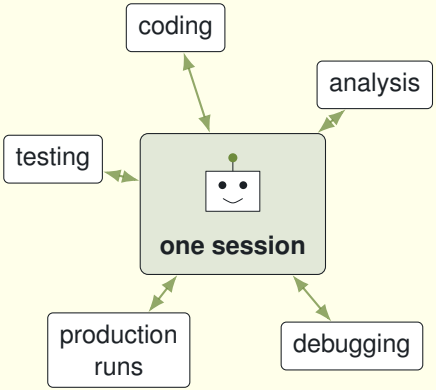
Bartlett '81 – Barry '81 Professor and Head
Department of Chemical and Biological Engineering
Drexel University
Philadelphia, Pennsylvania
cfa22@drexel.edu

Version 2.0 — 11 October 2026

View these slides
on your phone

Drexel UNIVERSITY

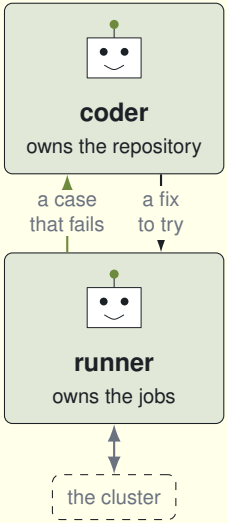
How it all started: one agent for all tasks



It worked. It did not scale — and the reasons became this talk:

- Everything shared one memory. A long run's output pushed out the bug I was chasing.
- While it ran the simulation, I could not use it to write code. I waited.
- One set of notes for every job: how to release the code, how to submit to the cluster, all in one file.
- It checked its own work. The session that wrote the mistake was the session reviewing it.

Two agents: one writes the code, one runs it

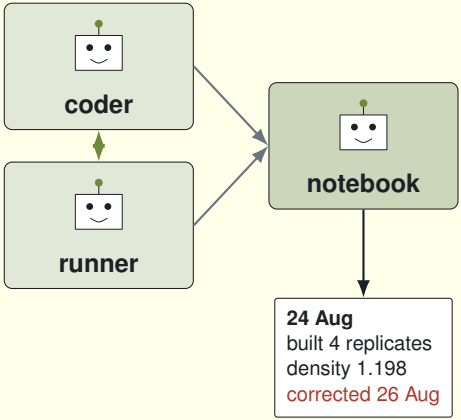


What the split fixed, in order:

- Two memories. The run's log stopped pushing out the code I was reading.
- They work at the same time. One waits on the queue; the other edits.
- Two sets of notes: how to release the code, and how to submit a job.
- **The one that finds the bug did not write it.**

Everything after this is that idea, repeated.

Three agents: and one of them writes it down

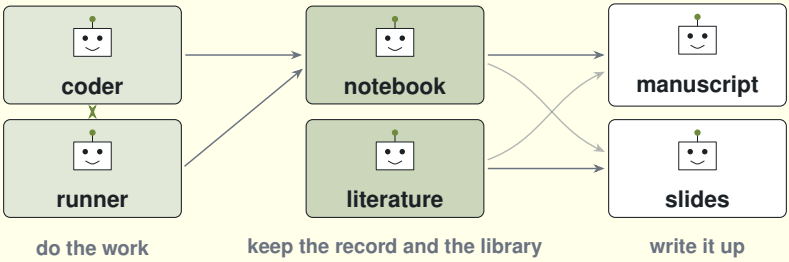


Now I could stop remembering things:

- What was run, and what came out, is written down — not held in a session that will end.
- I can read it in the morning instead of asking three agents what they did.
- It writes the correction *underneath* the claim. Nothing is quietly replaced.

Three sessions, three jobs — and a record none of them owns alone.

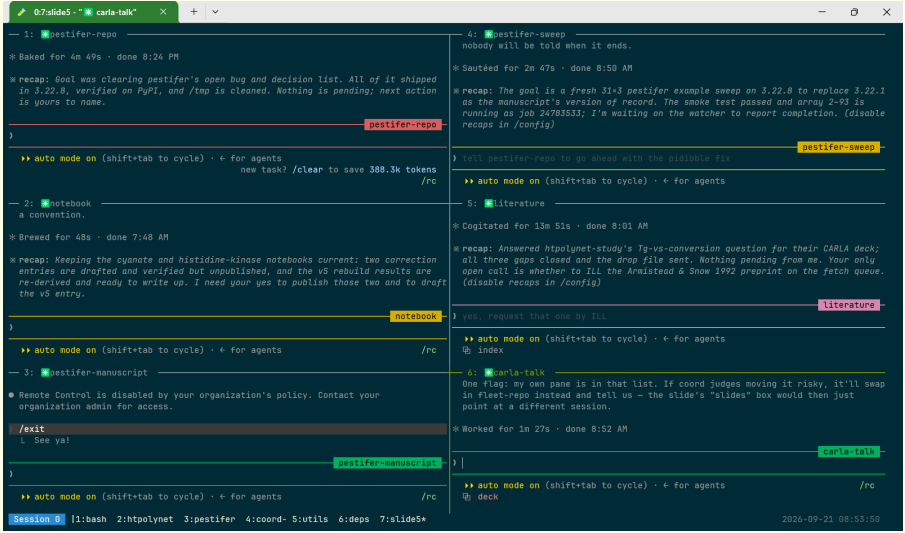
Six agents: doing, remembering, writing



- **literature** owns my paper library. The others ask it instead of searching the web themselves.
- **manuscript** writes from the notebook's record and the library's quotes.
- **slides** built the talk you are watching — these slides, in September 2026.

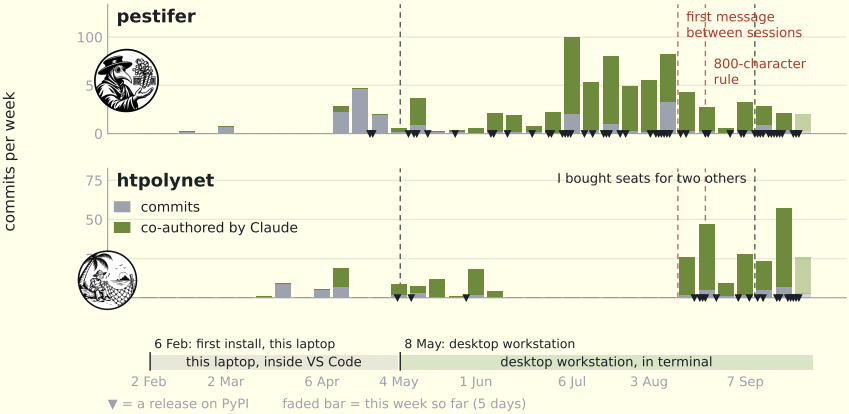
Six sessions, six jobs. Nobody holds two.

Six agents: doing, remembering, writing



The same six, 21 September, 08:53: one tmux window, one agent to a pane.

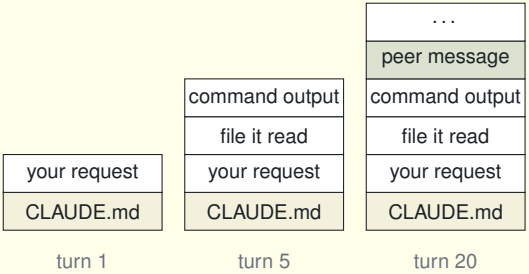
What changed in my pace — and the day they first talked



February to May: this laptop, inside VS Code. Then the desktop workstation, in a terminal — and by July, a hundred commits in one week.

Three in four carry a Claude co-author line. The July peak is one session, not the fleet.

Context: everything the agent can see



- Context = the instructions, the conversation, and every file and output so far.
- On every turn, the model reads the whole context again.
- It has a size limit. When it is full, old parts are summarized.
- A new session starts empty. Only files survive.

Keep this picture in mind: it is why messages between sessions are expensive.

CLAUDE.md: notes the agent reads every session

~/.claude/CLAUDE.md — how I work, read in every directory

```
Never use unscoped pkill/killall with a pattern. Kill by PID.
Detach long runs with setsid, and monitor via their log file.
```

my_package/CLAUDE.md *the same file* **my other codes**

```
uv run --extra test
pytest tests -q
# NOT tests/unit
```

```
Release with
scripts/release.sh.
Never hand-roll it.
```

```
...one file
in every
directory
```

When the agent makes a mistake, the fix goes into one of these files.

Try this: one CLAUDE.md per directory you work in. Add a line each time the agent makes a mistake.

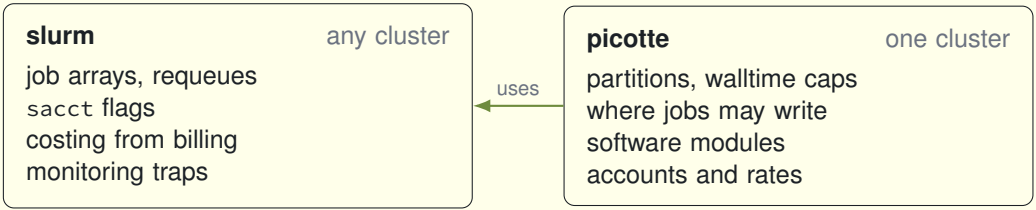
What I kept re-explaining, written down once

skills/slurm/SKILL.md, first lines (abridged)

```
---
name: slurm
description: General SLURM mechanics and the traps that produce
  confident wrong answers ... Triggers on "sbatch", "sacct",
  "squeue", "job array", "requeue", "NODE_FAIL", "billing".
---
```

- The agent sees only the name and description.
- When a task matches, it loads the whole file.
- CLAUDE.md is per directory. A skill follows the *task*.

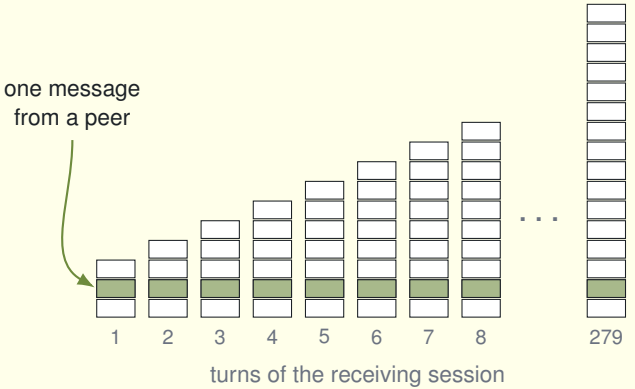
One file for any cluster, one for mine



- Another site can take **slurm** and write its own site file.
- Account names and rates stay in the private site file.

Try this: write a skill for your own cluster. Start with the three mistakes you see most.

The bill for talking: a message is read again on every turn



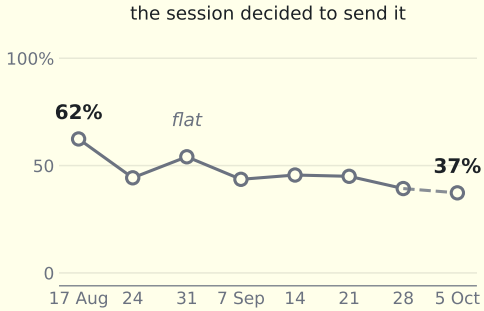
367×

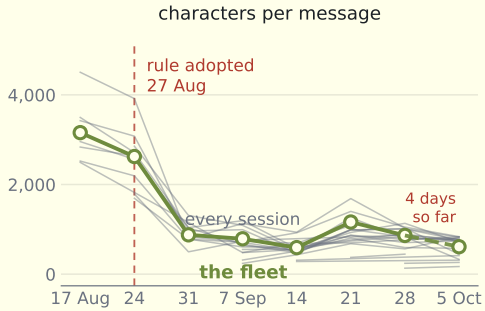
written once: 1.2 M tokens
read again: 443 M tokens

Rule: write the content to a file. Send the path and one line.

Measured: this costs about **30%** of sending the file itself — because the reader reads only the part it needs.

What the sessions learned: how to write, not whether



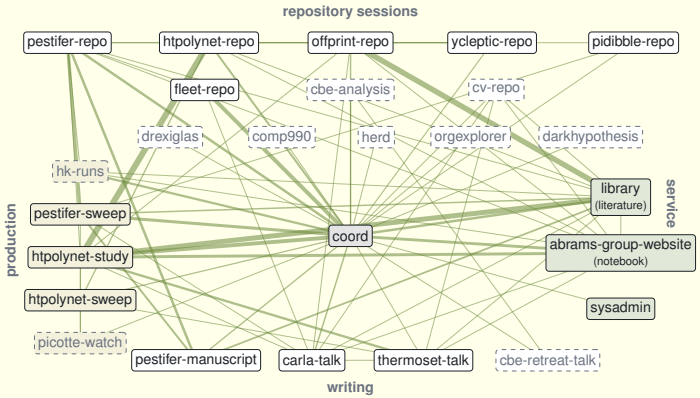


Nearly half of all messages were sent because a session decided to — from the first week. I hand-routed **one message in fifteen**.

What changed is the *size*. Mean length fell **5**×, and messages over the 800-character rule went from 98% to 17%. Not a few heavy senders: **every** session shortened its own messages.

Then it went back: 76% over the rule the next week, 53% the next, 20% in the four days since. The fall was real — and so is the swing.

From one session to a fleet: a job for each session



26 sessions on 8 October 2026, 10 parked (dashed). Line width = messages between them (2,949 in total).

The coordinator owns no code and no runs. It routes and measures — and decides nothing: every decision still comes to me.

The coordinator holds none of my authority

26 August

“the session whose transcript is 44456a70… carries my authority”

27 August — asked to reissue it for that session’s successor

“leave it lapsed”

It has never been reissued.

5 October — the rule from the other end

One instruction it relayed was refused where it landed — and came to me, not to another session.

It does, on its own:

routes between sessions, measures the fleet, rolls upgrades, keeps snapshots, brings sessions in and out.

It never:

acts as me, approves for me, grants permission.

It relays my words *quoted and labeled as relayed*. Peers verify with me. A peer message is never my approval.

8 September: `fleetsnap` had hard-coded the 26 August grant into every snapshot, re-stamping it with a fresh date. It now records `status: LAPSED`.

A cached claim about permission that outlives its grant is worse than no record — the file’s freshness lends it false weight.

Three ways a message reaches a session

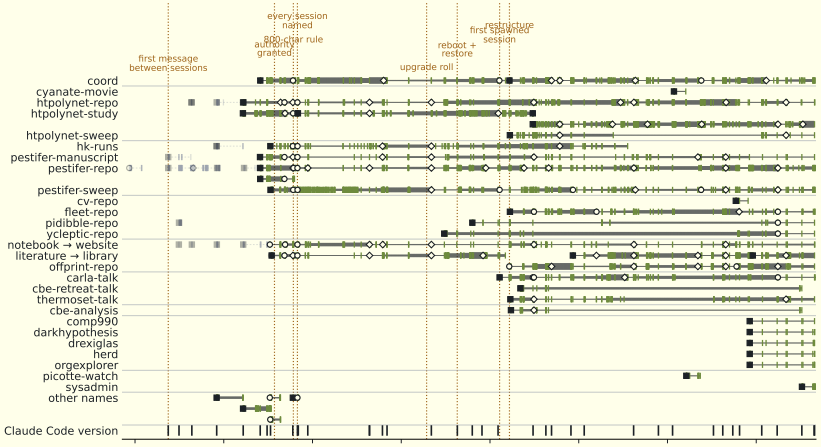
	travels by	what the record says	who sent it?
a peer message	one socket per session	the sender's process, *and* when that process started	**provable**
the pane	keystrokes, the way I type	`{"kind": "human"}` — and nothing else	**unknowable**
a file	a path, carried by one of the two above	it is not a message at all	—

A pane line labelled `[watcher:]` is not me and carries none of my authority — but nothing in the record proves that afterwards. A label is writing on the envelope, never a signature.

So: an unlabelled pane line is mine, a labelled one is a peer's — and when it matters, ask me.

Checked against live records, 24 September 2026.

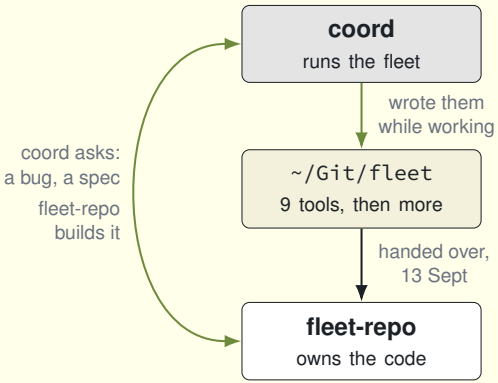
Every session in the fleet, 14 Aug – 7 Oct



Thickness = context carried, to scale (the fattest are near 1 M tokens) · ticks: 15 min active · ■ start ○ `/clear` ◇ summarized

faded: no name recorded, placed by directory, drawn thin · *other names*: not placed in a lane

The fleet grew its own software



- A session built tools to do its own job. The tools became a repository with its own session.
- The rule that followed, in the coordinator's own notes:
  "coord no longer edits `~/Git/fleet`*"*
- The author became a user. It asks; it does not patch.

The paper library did the same, this week: the session that had built the mirror tool kept the code (**offprint-repo**), and a **new literature session** runs the library. It files bugs; it does not patch.

Three of those tools, on a normal morning

`fleetspawn` *name dir*
Makes the pane, labels it, starts the session on its brief. It refuses to start if the brief, the label or the name is wrong — and changes nothing when it refuses.

`fleetupgrade`
Who is running an old binary, and has each one written down what it is watching? Then an order to restart them in.

`fleetcontext`
How much each session is carrying, and which one to trim. It recommends; I choose.

```
$ fleetcontext
  compact above 250k, clear above 500k

  SESSION              CONTEXT  STATE  DO
  htpolynet-repo          651k  idle   clear?
  htpolynet-study         540k  idle   later
  coord                   514k  idle   manual
  carla-talk              424k  busy   manual
  hk-runs                 330k  idle   later
  pestifer-sweep          254k  idle   compact
  notebook                248k  idle   leave
  literature               10k  idle   leave
```

A real run, 20 September. The red line is the session making these slides.

The host underneath the agents

Six things, or there is no fleet

`tmux`	every session is a pane
`git`	every session commits
`python3`	the fleet tools
`bash`	+ grep, sed, awk, find
`ssh` + key	to reach the cluster
`systemd --user`	watchers that outlive it

Then: `uv`, `gh`, `curl`, `rsync`. An afternoon's work.

What surprised me

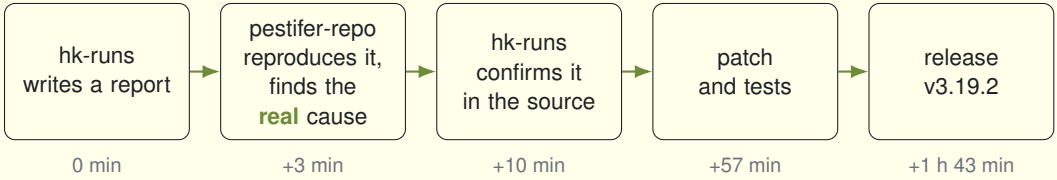
No cluster software at all. `sbatch`, `sacct`, `apptainer`: zero calls on this machine, about 1,300 inside `ssh`. The host needs an account, not a copy of the cluster.

A machine set up only for simulation would fail here. The simulation tools run locally only for *tests* — the real runs are on the cluster. What runs here every day is TEX, `pdftoppm`, `pdftotext`, ImageMagick: ∼**1,600** calls.

Reading papers and making documents *is* the local work.

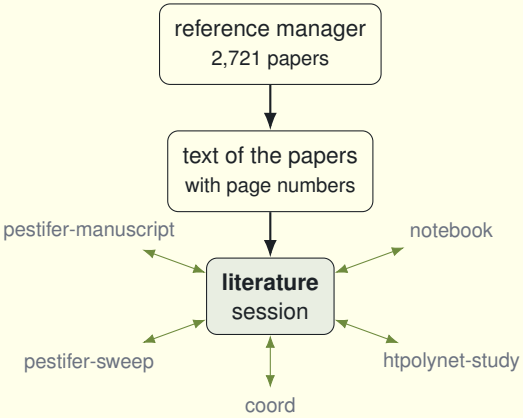
One bug, one afternoon

Symptom. A production build asked for protein chain A only. Pestifer built chains A *and* B. No warning, exit status 0. The “mutant” was mutated on one chain only.



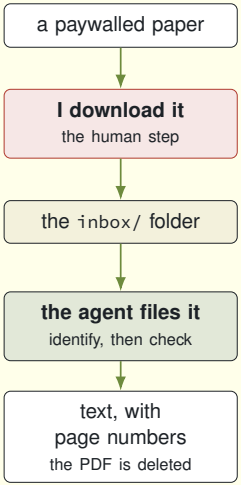
- The production session blamed the wrong function. The repository session found the real one by reproducing the bug.
- Six runs had already used 35 GPU-days. The native-vs-mutant comparison was void.
- The two sessions talked directly. The coordinator was not in this path.

The literature agent



- Mirrors my whole reference library as searchable text, page by page.
- Other sessions ask it questions. They do not search the web themselves.
- Answers come with a quote and a real page number.
- About 350 messages with 8 other sessions.

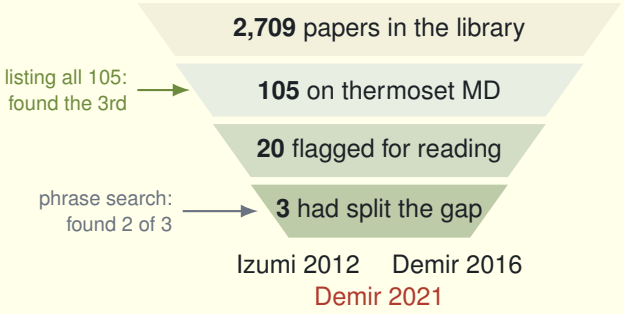
How a paywalled paper gets in



- The agent does not work around a paywall. The rule in its notes: *“don’t script a way past a paywall.”*
- It names the paper from the file name, the file itself, or the first page.
- It attaches the file only if that title is really on page 1. If not, the file stays in the folder.
- A scan has no text: 109 were read by letter recognition. A report with no DOI gets a short description file, written by hand.

“I cannot fetch this” is a real answer. Then I request the paper from another library.

The literature agent at work: “nobody has done this”



Simulated thermosets are ∼2% less dense than real ones.

A draft said: nobody has split that gap into a monomer part and a curing part.

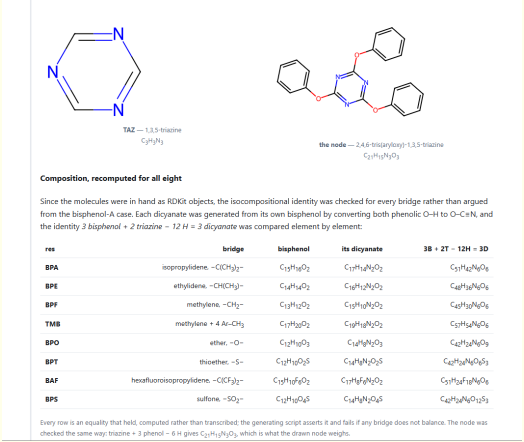
A phrase search can confirm. Only a full list can support “nobody”.

The notebook agent

- Keeps the group's lab notebook: members-only web pages.
- Checks other sessions' results against the logs and files before it writes.
- About 250 messages with other sessions.

Its rules

- **Append only.** A wrong number stays, with the correction below it.
- **Check before publishing.** A result is a claim, not a fact.
- **Date every measurement.**



TAZ — 1,3,5-triazine
$C_3H_3N_3$

the node — 2,4,6-tris(aryloxy)-1,3,5-triazine
$C_{21}H_{15}N_3O_3$

Composition, recomputed for all eight

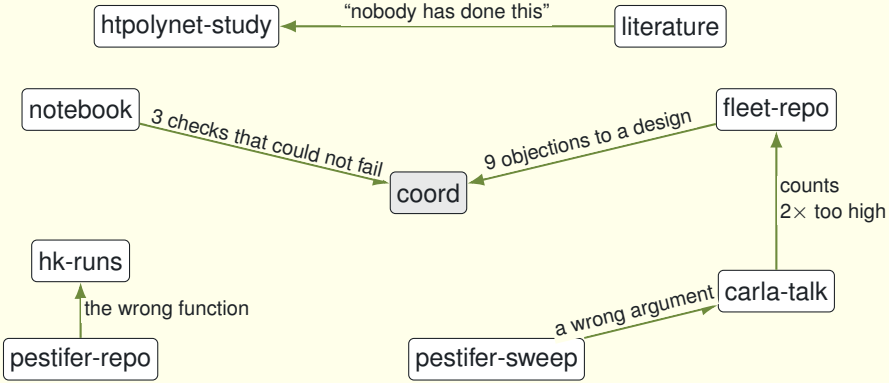
Since the molecules were in hand as RDKit objects, the isocompositional identity was checked for every bridge rather than argued from the bisphenol-A case. Each dicyanate was generated from its own bisphenol by converting both phenolic O–H to O–C≡N, and the identity *3 bisphenol + 2 triazine − 12 H = 3 dicyanate* was compared element by element:

res	bridge	bisphenol	its dicyanate	3B + 2T − 12H = 3D
BPA	isopropylidene, –C(CH$_3$)$_2$–	$C_{15}H_{16}O_2$	$C_{17}H_{14}N_2O_2$	$C_{51}H_{42}N_6O_6$
BPE	ethylidene, –CH(CH$_3$)–	$C_{14}H_{14}O_2$	$C_{16}H_{12}N_2O_2$	$C_{48}H_{36}N_6O_6$
BPF	methylene, –CH$_2$–	$C_{13}H_{12}O_2$	$C_{15}H_{10}N_2O_2$	$C_{45}H_{30}N_6O_6$
TMB	methylene + 4 Ar–CH$_3$	$C_{17}H_{20}O_2$	$C_{19}H_{18}N_2O_2$	$C_{57}H_{54}N_6O_6$
BPO	ether, –O–	$C_{12}H_{10}O_3$	$C_{14}H_8N_2O_3$	$C_{42}H_{24}N_6O_9$
BPT	thioether, –S–	$C_{12}H_{10}O_2S$	$C_{14}H_8N_2O_2S$	$C_{42}H_{24}N_6O_6S_3$
BAF	hexafluoroisopropylidene, –C(CF$_3$)$_2$–	$C_{15}H_{10}F_6O_2$	$C_{17}H_8F_6N_2O_2$	$C_{51}H_{24}F_{18}N_6O_6$
BPS	sulfone, –SO$_2$–	$C_{12}H_{10}O_4S$	$C_{14}H_8N_2O_4S$	$C_{42}H_{24}N_6O_{12}S_3$

Every row is an equality that held, computed rather than transcribed: the generating script asserts it and fails if any bridge does not balance. The node was checked the same way: triazine + 3 phenol − 6 H gives $C_{21}H_{15}N_3O_3$, which is what the drawn node weighs.

A notebook page. Every formula in the table was computed and checked by a script, not typed. 3 notebooks so far, 62 dated entries.

Why it worked: the catcher owned the thing



Arrow: *caught a mistake in*. Not because it was smarter: it *owned* the thing the claim was about.

The error and the review share a blind spot.

The traps I fell into

three of them, all mine

A check that cannot fail is not evidence

What Claude tried first

```
$ pytest tests/unit -q 2>&1 | tail -1
no tests ran in 0.00s
$ echo $?
0
```

“Empty test suite. Fine.”

The same run, not piped

```
$ pytest tests/unit -q > out 2>&1; echo $?
4
$ head -1 out
ERROR: file or directory not found: tests/unit
```

Wrong path. `tail` cut the error; `$?` was `tail`’s.

Before you write “verified”, say what result would have shown it was wrong.

Real output, reproduced 13 Sept.

These checks all fail in the same direction

The check	It said	The truth
wait until `pgrep -f pattern` finds nothing	✓ still running	× it found itself: 5 days
which processes watch this job?	✓ watched	× the coordinator only looked
the analysis log is empty	✓ never ran	× 32 of 32 builds done
a process runs in the right folder	✓ session is up	× waiting at “trust this folder?”

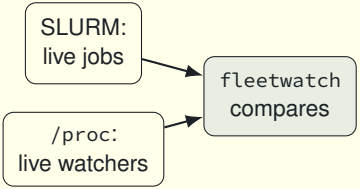
When they are wrong, they say “fine”.

Records kept by hand go out of date

watchers, written by hand

job A ..... watched
job B ..... watched
(the job nobody wrote down)

looks complete.
It cannot show what is missing.




```
23472564  21 RUNNING -> *** NOBODY ***
```

You cannot record the monitor you forgot to create.

Honest costs

My attention

4.8 days

one session waited that long for my answer

Tokens

32.3 billion

tokens read since mid-August — 97% of them re-reads

Upkeep

40 versions

of Claude Code in eight weeks

Tooling

10,703 lines

of tools, skills and notes, just to run the fleet

And agents are sometimes wrong and sure of it — including the coordinator. Every claim needs a check that can fail.

When is a fleet too much?

One session is enough when:

- you work in one repository at a time
- your runs are short, and you watch them yourself
- nobody else depends on what the agent produces

A fleet starts to help when:

- production runs find bugs in code that another session owns
- cluster jobs run for days and need a watcher
- you want a second agent to check the first one's claims

And even then: "we need to simplify." — me, to the coordinator, 12 Sept.

Here is what has me worried

I am training my students differently

I learned this work by writing every line myself, and by being wrong slowly.

They are learning to direct an agent and to check what it gives back. That second skill is real, and they are faster than I was.

But I do not know yet what is lost — and I am deciding for them.

Not everyone can buy this

Since mid-August my sessions read **24.6 billion** tokens. That runs on a paid subscription, on a machine that is always on, beside a cluster.

I bought **three seats**: me, a student, a postdoc. That was a choice I was able to make.

A group that cannot make it is not one step behind. It is on a different curve.

The cheapest parts of all this are the ones that helped me most — and they are free.

What I would tell you to try

Now. One repository, one `CLAUDE.md`. Add a line each time the agent gets something wrong.

Within a week: a pair. One agent that owns the *code*, one that owns the *runs*. Let the runner report the bug and the repository fix it. That pair found my chain-building bug and released the fix in **1 h 43 min**.

Within a month: an agent that owns a shared thing. Your papers, or your lab notebook. Everyone else asks it instead of searching alone — and it is the one that catches you.

One command: `fleetspawn` *`name dir`* — pane, brief, session. It is in the fleet repo.

And one rule, whatever you do: before you accept "verified", ask what result would have shown it was wrong.



the tools, free to take

`github.com/cameronabrams/fleet`

`fleet-of-agents.readthedocs.io`

Questions?

`cfa22@drexel.edu`

The code behind all of this



pestifer pypi v3.26.0

NAMD systems from a PDB entry.



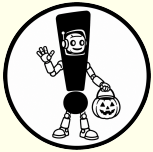
htpolynet pypi v2.15.0

Amorphous polymer networks.



pidibble pypi v1.12.2

A complete PDB file parser.



ycleptic pypi v2.4.4

YAML configuration schemas.



offprint github v0.31.0

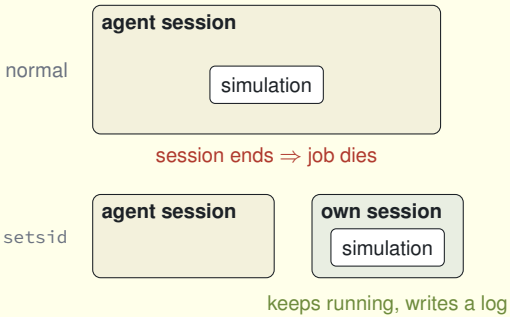
A plain-folder paper library.



fleet github v0.2.0

Runs the fleet of sessions.

Backup: why `setsid`?



A job started normally belongs to the agent's session. When the session ends, the job gets `SIGHUP` and stops.

`setsid` gives the job its own session, with no terminal. Ending the agent session no longer stops it.

```
setsid bash -c 'echo $$ > run.pid; exec ./run.sh' > run.log 2>&1 < /dev/null &
tail run.log                    # watch the log, not the agent's memory
kill -- -$(cat run.pid)         # stop exactly this job and its children
```

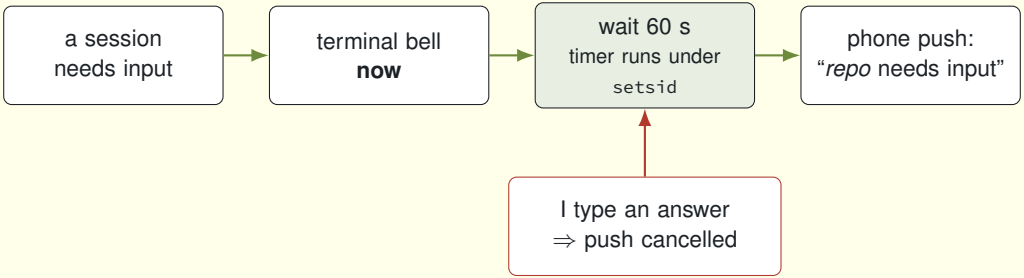
Backup: why a local library, not the whole web?

The world is where it looks. The library is where it can check.

- **Paywalls.** The web gives abstracts; the numbers are in the tables.
- **Summaries, not sources.** A summarizer rewrote a patent's tables; the numbers had to be read again.
- **Some evidence is only an image.** A 1995 review's key figure has no usable text.
- **Only a finite set can be listed.** "Nobody has done this" needs every paper. The web only allows "I did not find it."
- **Pages stay put.** A quote keeps its page number, so wrong page numbers can be caught.
- **It is the group's library.** Same citation keys as our manuscripts.

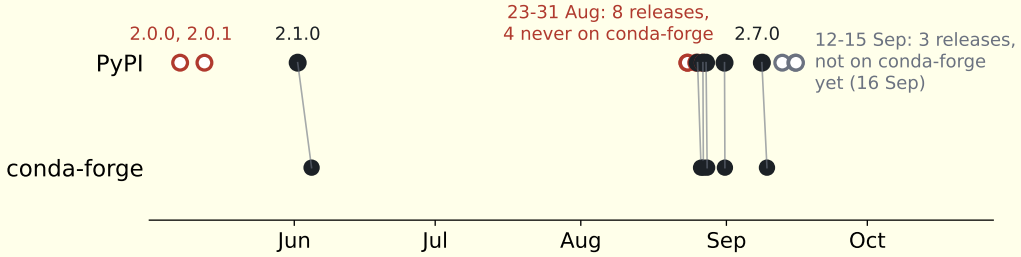
Use both: search the world, and file what can be obtained, so the next question starts from a page.

Backup: how the fleet reaches my phone



- Only sessions launched as fleet agents may notify. A `claude` typed by hand stays quiet.
- Mute everything, one session, or make the wait longer — checked when the timer ends.
- This week: new sessions carried the flag and asked many first-day questions — and my phone suddenly got many pushes.

Backup: conda-forge — a release can fail silently



htpolynet: every release on PyPI (top) and on conda-forge (bottom). Red open circles never reached conda-forge; gray ones have not yet.

Every claim in the skill carries a tag and a date: `OBSERVED` · `REPORTED` · `UNTESTED`

Backup: inside the slurm skill — traps, each with evidence

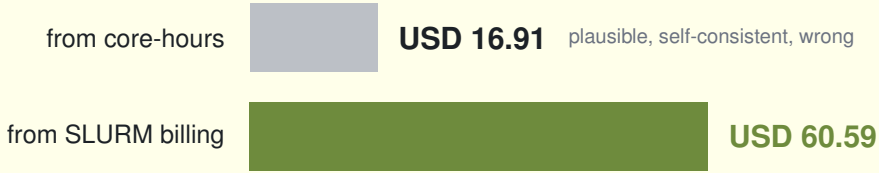
Each of these checks, when wrong, says “fine”.

Question	needs `sacct -D`?	Why
What did this cost?	yes	requeued runs are billed
Should I resubmit this failure?	yes	SLURM may have requeued it already
Is the job finished?	no	old NODE_FAIL rows look permanent

Observed: 8 rows without `-D`, 11 with it. Three “failed” tasks were already running again.

Backup: a confident wrong number

A GPU job. How much did it cost?



The GPU partition bills CPUs at zero. Read `billing=` from `AllocTRES`.

Backup: rules the literature agent earned

Name the tools. Do not claim the family.

“No tool of this kind does X”:



Held for five papers. Broken by the sixth and the seventh — while already in a manuscript.

Use the issue date, not the online date.

PUBLISHED ONLINE 7 NOVEMBER 2016

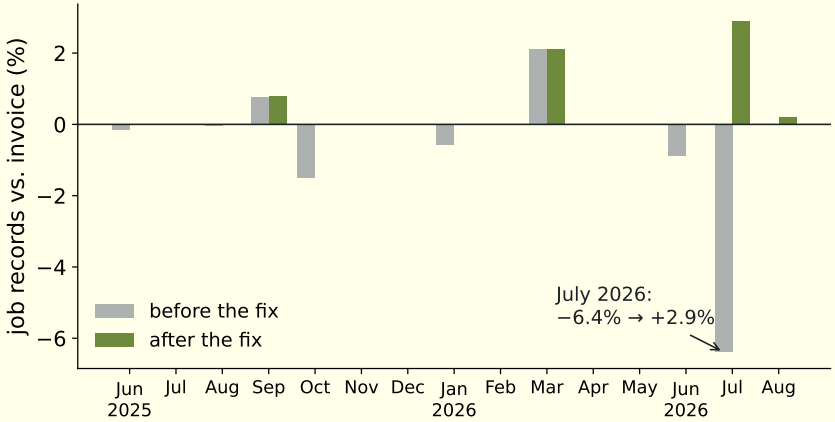
the online date: what the metadata uses → filed as 2016

NATURE METHODS | VOL.14 NO.1 | JANUARY 2017 | 71

the issue, on the same page → cite as 2017

CHARMM36m had been cited as 2016. The fix went into the tool.

Backup: the notebook agent at work — a bill it rebuilt



15 months of cluster invoices vs. SLURM job records. After the fix, 11 of 15 months match exactly. On the way, it found its *own* parser dropping rows.

Backup: the notebook agent at work — a claim it withdrew

24 August 2026 — a correction to my own multiplier, and an unresolved density offset

source	BADCy density at χ ≈ 0.90–0.92, 20 °C	our offset
this work, mean of 4 replicates	1.1983 g/cm^3 (sd 0.0037)	—
Snow 1991, mercury dilatometry, χ = 0.91	1.205	−0.0067 = **0.56%**
Reams 2012, flotation, χ = 0.92	1.197	+0.0013 = **0.11%**

⋮

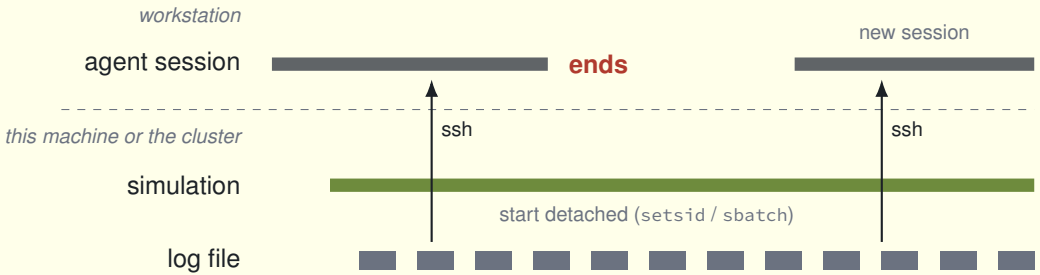
13 entries later, on the same page

26 August 2026 — the density agreement was on the wrong axis, and the correction goes the wrong way

Matched correctly, the notebook wrote, the model was 1.40% and 1.73% low.

Rebuilt since, with two charge bugs fixed: about **1.1%** low (21 Sept). Every version stays on the page.

Backup: long runs — the agent is not the job



the agent reads the log, not its own memory of the run